Part B Report

1993 words

Introduction

This is a genetic algorithm project that evolves creatures to climb a mountain in a PyBullet physics engine. This project evaluates creatures in a simulation, selects parents based on fitness, creates offspring via crossover and mutation, and preserves the best individual for the next simulation.

In this paper, we will study how population size and generation count affect creature evolutions. We will also analyze how different mutation types affect creature complexity, and how motor patterns improve performance.

Method

Fitness function

Fitness function is evaluated based on the Euclidean distance between the creature's last position and the top of the mountain. In this study, a **lower fitness value** equals better fitness as it means that the creature has **climbed closer to the mountain summit**. A value of 0 would mean that the creature has reached the summit, and achieved maximum fitness.

```
creature.py > Creature > get_distance_to_top
 34   class Creature:
120       def get_distance_to_top(self, mountain_position=(0, 0, -1), mountain_height=5):
122           Distance from creature to top of mountain.
123           """
124           if self.last_position is None:
125               return float('inf')  # If creature hasn't moved, return infinite distance
126           p1 = np.asarray(self.last_position)
127           # top of mountain
128           mountain_top = np.asarray([mountain_position[0], mountain_position[1], mountain_position[2] + mountain_height])
129           dist = np.linalg.norm(p1 - mountain_top)
130           return dist
```

This function takes the last position of the creature and uses the 'linalg.norm()' function from NumPy to calculate the Euclidean distance travelled by the creature.

$$d(x,\ y)\ =\ \sqrt{\sum_{i=1}^{n}(y_i - x_i)^2}$$

This function gets the last position of the creature using self.last_position, and the position of the mountain. The mountain position is established as (x, y, z), where z is the height. Through prepare_shapes.py, we discover that the height of the mountain is 5. As such, if the mountain is initialized at position (0, 0, 0), the top of the mountain would be as (0, 0, 5)

By choosing the destination point to be (0, 0, 5) rather than (0, 0, 0), we would be able to evaluate the creature's ability to traverse horizontally and vertically.

Linalg.norm takes the creature's last position, subtracted by the position of the mountain top, and calculates the Euclidean distance between the two points. This gives us the fitness value.

In the respective test_ga.py files, I altered the code so that the algorithm would preserve the best creature based on the new fitness value. In this case, creatures that had travelled closest to the mountain top would be saved.

```
14   class TestGA(unittest.TestCase):
15       def testBasicGA(self):
49               # elitism
50               max_fit = np.max(fits)
51               for cr in pop.creatures:
52                   if cr.get_distance_travelled() == max_fit:
53                       new_cr = creature.Creature(1)
54                       new_cr.update_dna(cr.dna)
55                       new_creatures[0] = new_cr
56                       filename = "elite_"+str(iteration)+".csv"
57                       genome.Genome.to_csv(cr.dna, filename)
58                       break
```

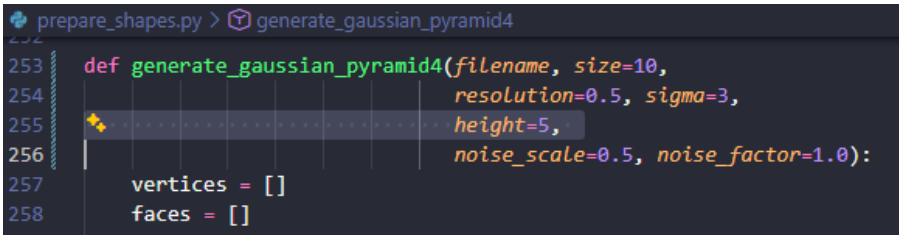
Original code

```
25   class TestGA(unittest.TestCase):
26       def testBasicGA(self):
85               # elitism -- preserve best individual (lowest distance to top)
86               best_fit = np.min(fits)
87               for cr in pop.creatures:
88                   if cr.get_distance_to_top() == best_fit: # finds the best fitness according to distance to mountain
89                       new_cr = creature.Creature(1)
90                       new_cr.update_dna(cr.dna)
91                       new_creatures[0] = new_cr
92
93                       # save in elite folder
94                       ga_dir = os.path.dirname(os.path.abspath(__file__))
95                       elite_dir = os.path.join(ga_dir, "elite")
96                       os.makedirs(elite_dir, exist_ok=True)
97
98                       filename = os.path.join(elite_dir, "elite_"+str(iteration)+".csv")
99                       genome.Genome.to_csv(cr.dna, filename)
100                      break
```

Updated code

Environment

This study uses the basic gaussian pyramid model. From prapare_shapes.py, I obtained the height of the model. It is used for the fitness function method below.



Creature

The creature from the source code is used in this study and a backup copy is kept. The creature evolves with each experiment, and is manually reset for every new experiment.

Creatures are represented by structures composed of links and joints. Each creature's genome encodes link dimensions, joint types, and control parameters, as seen in genome.py().

Selection method

Roulette wheel selection is used for parent selection. It is a stochastic selection method, meaning that the probability of selection is proportional to the fitness value.

Initial parameters

- Population size: 20
- Gene count: 3
- Time steps: 2400
- Point mutation rate: 0.1
- Grow mutation rate: 0.25
- Shrink mutation rate: 0.25
- Number of generations: 20

Basic exploration

In this section, we will experiment with different parameters and observe how they alter the results of the experiment.

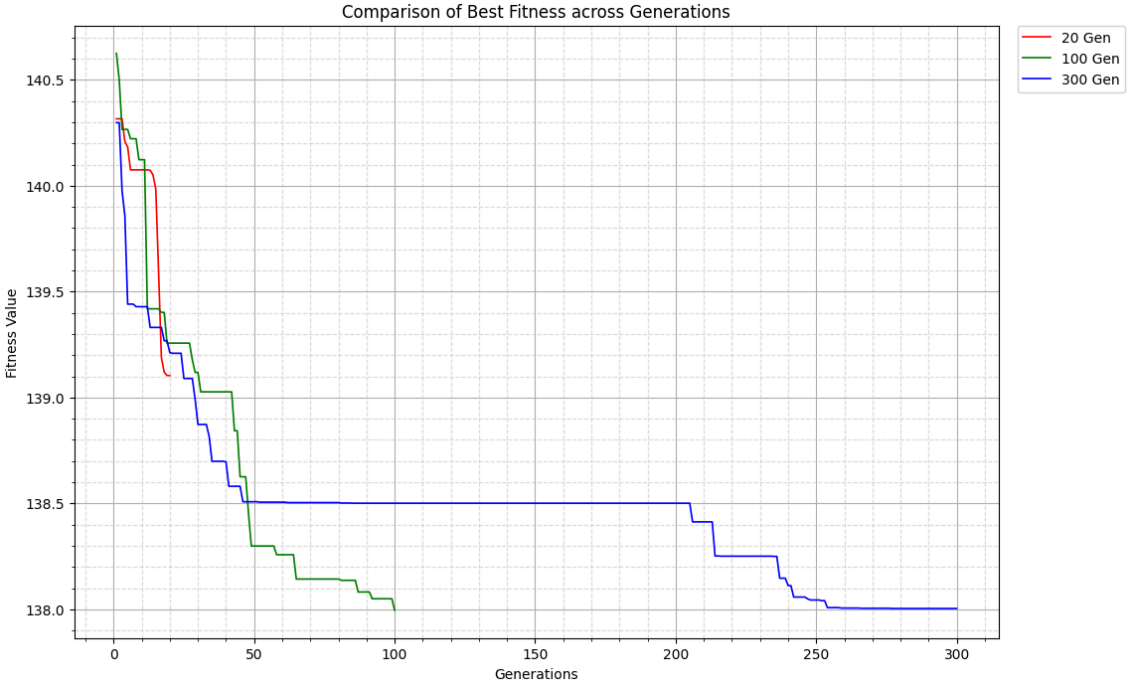
Number of generations

In this section, we compared the outcomes across 20, 100, and 300 generations while holding all other parameters constant. Below is a table and a best fitness graph comparing the results.

Generations	Best fitness	Mean fitness	Max links	Mean links
20 (Baseline)	139.104	140.522	31	5.05
100	137.996	140.325	121	9.85
300	138.004	140.229	58	5.01

In the graph, we observe that there is an exponential decay of best fitness as the generation increases. Across all three experiments, we can observe that the more generations run, the closer the creature gets to the top of the mountain. There is an 0.8% improvement in best fitness from 20 to 300 generations.

There is also an improvement in mean fitness, as seen from the decreasing value from 140.522 in 20 generations to 140.299 in 300 generations. This indicates that the creatures that have evolved through more generations are more likely to travel closer to the top of the mountain if more generations are run.





Unexpectedly, the 100-generation run produced the highest number of maximum links at 121, while the 300-generation run produced the highest of 58. This is because the algorithm does not penalize creature complexity. This allows the algorithm to create more links for exploration.

Similarly, the 300-generation run produced the lowest mean links across all generations at 5.01 links, a decrease from 5.05 links in the 20-generation run. This tells us that the creature complexity is non-linear with the number of generations run.

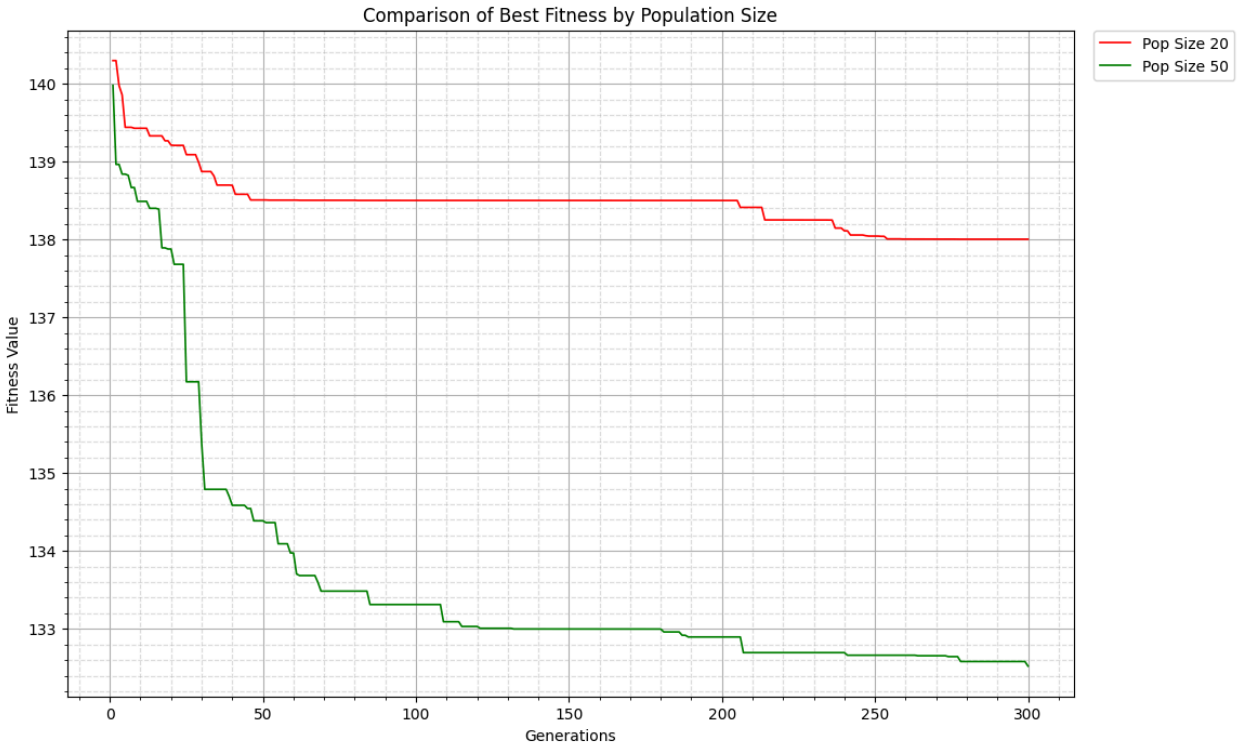
We can thus conclude that running more generations would allow creatures to evolve into ones that can better scale the mountain. However, it is not indicative of more complex creatures.

Population

We will also compare the fitness values following changes in population size. This experiment is run for 300 generations, and other parameters are kept constant.

Population size	Best fitness	Mean fitness	Max links	Mean links
20	138.004	140.229	58	5.01
50	132.522	inf	445	42.24

While we were able to attain a mean fitness value for the 20-generation run, a ‘inf’ value was recorded for the 50-population run. This can be caused by overly complex creatures resulting in engine errors and timeout failures.



Both populations show a rapid drop in fitness value in the first 50 generations, with the 50-generation run showing a sharper decline.

The 20-population size run hits a plateau that lasts from generation 50 to 200, suggesting the smaller population size limits further exploration and that it is dependent on a lucky mutation to make further progress.

Meanwhile, the 50-population size run demonstrates consistent improvements throughout 300 generations. This is because the larger population size allows for more exploration, increasing the probability of discovering beneficial mutations that achieves better fitness values.

We also observe a dramatic increase in the maximum number of links recorded, from 58 in the 20-population size run to 445 in the 50-population size run. This suggests that larger populations enable the exploration of highly complex creatures.

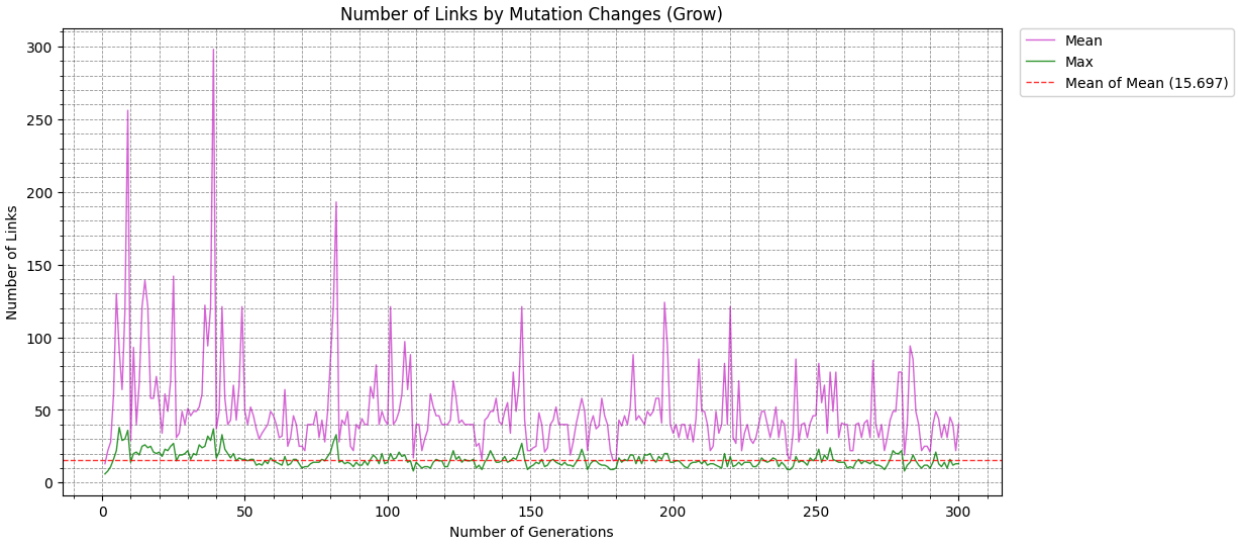
It is also noted that while a larger population size allows for better fitness values, it requires more computational power and time to support the more complex creatures. Running the simulation often caused the program to suddenly exit prematurely.

Mutation

Next, we compare the results when we increase the mutation values. We will run these experiments for 300 generations. The table and graphs below depicts the number of links against the different mutation changes.

	Grow mutation	Point mutation	Shrink mutation	Best fitness	Mean fitness	Max links	Mean links
Baseline	0.1	0.1	0.25	138.004	140.229	58	5.01
Grow	0.5	0.1	0.1	137.864	inf	298	15.69
Point	0.1	0.5	0.1	138.362	140.558	67	6.43
Shrink	0.1	0.1	0.5	138.141	140.314	40	2.85

Grow mutation

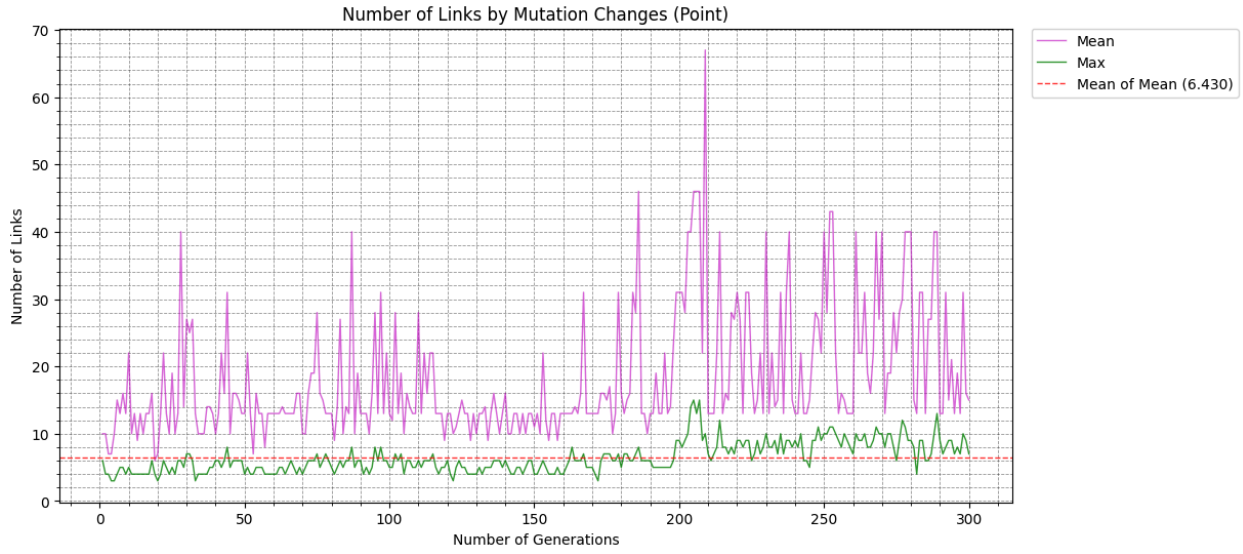


The growth mutation is the most volatile as increasing the growth mutation adds complexity to our creatures. It has the highest mean number of mean links at 24.34~, hitting a maximum at 298 links in generation 39.

A slight improvement in the best fitness value is also observed. This suggests that growth mutation allows for exploratory search by increasing creature complexity, which in turn improves best fitness slightly.

Another observation is that with a 0.5 growth mutation, the computational time to run all 300 generations is significantly longer than other mutation changes.

Point mutation

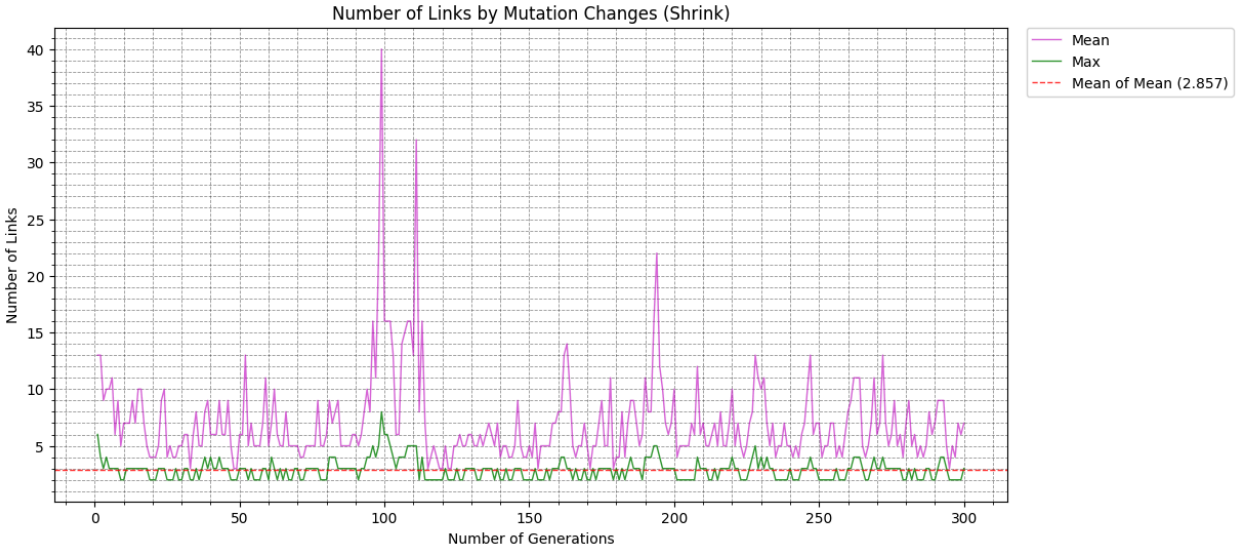


Point mutation shows an equally volatile graph. However, its mean and maximum number of links is lower than that of growth mutation but still higher than that of the controlled results, with a maximum of 67 links and a mean of 6.43.

This tells us that point mutation helps prevent the creature from becoming overly mutated by the growth mutation as it allows for achieving a good fitness value without compromising computation power.

Shrink mutation

On the other hand, shrink mutation experiences a less volatile graph, with the mean number of mean links ranging from 2 to 8. This gives us a mean of 2.85~. At the same time, its maximum number of links is 40, lower than that of other mutation changes. This tells us that like point mutation, shrink mutation prevents the creature from over-mutating.

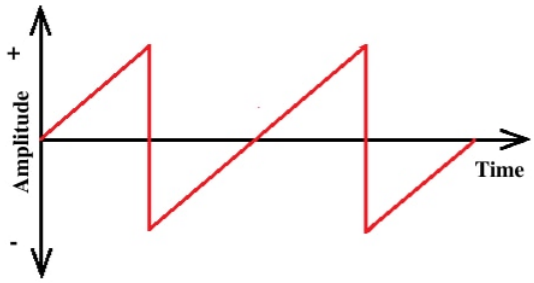


Mutation conclusion

From this, we can infer that the growth mutation allows for exploring more complex solutions while point and shrink mutations are responsible for preventing the creatures from becoming excessively large too quickly.

The baseline configuration of 0.1/0.1/0.25 grow/point/shrink mutation appears to be well-tuned for creature exploration and exploitation.

Advanced exploration



In this section, we will experiment with the motor controls. We will add a third motor type, Sawtooth. Sawtooth is a waveform that increases linearly before suddenly dropping.

First, I added the motor type into [creature.py]. Then I adjusted the Motor class to include the new control waveform conditions.

```
creature.py > Motor > get_output
6   class MotorType(Enum):
7       PULSE = 1
8       SINE = 2
9       SAWTOOTH = 3
10
11  class Motor:
12      def __init__(self, control_waveform, control_amp, control_freq):
13          if control_waveform < 0.33:
14              self.motor_type = MotorType.PULSE
15          if control_waveform < 0.66:
16              self.motor_type = MotorType.SINE
17          else:
18              self.motor_type = MotorType.SAWTOOTH
```

In the get_output() function, I defined the output logic to be (phase/pi) - 1. This produces an output ranging from -1 to 1 with sharp discontinuities. This allows a more jerky and sharp movement pattern that is different from the other motor types.

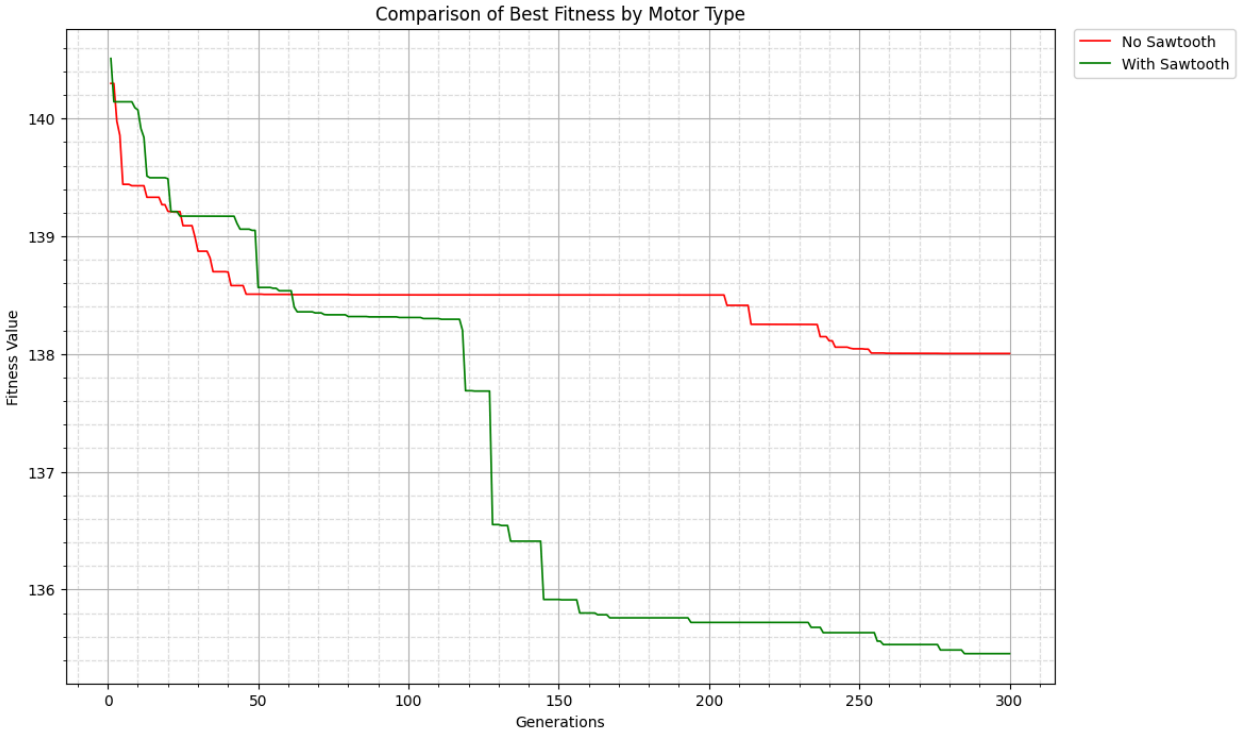
```
creature.py > Creature > to_xml > robot_tag
11  class Motor:
30      def get_output(self):
31          self.phase = (self.phase + self.freq) % (np.pi * 2)
32          if self.motor_type == MotorType.PULSE:
33              if self.phase < np.pi:
34                  output = 1
35              else:
36                  output = -1
37
38          if self.motor_type == MotorType.SINE:
39              output = np.sin(self.phase)
40
41          elif self.motor_type == MotorType.SAWTOOTH:
42              output = (self.phase/np.pi) - 1.0
43
44          return output
```

I will run this experiment across 300 generations at population size 20.

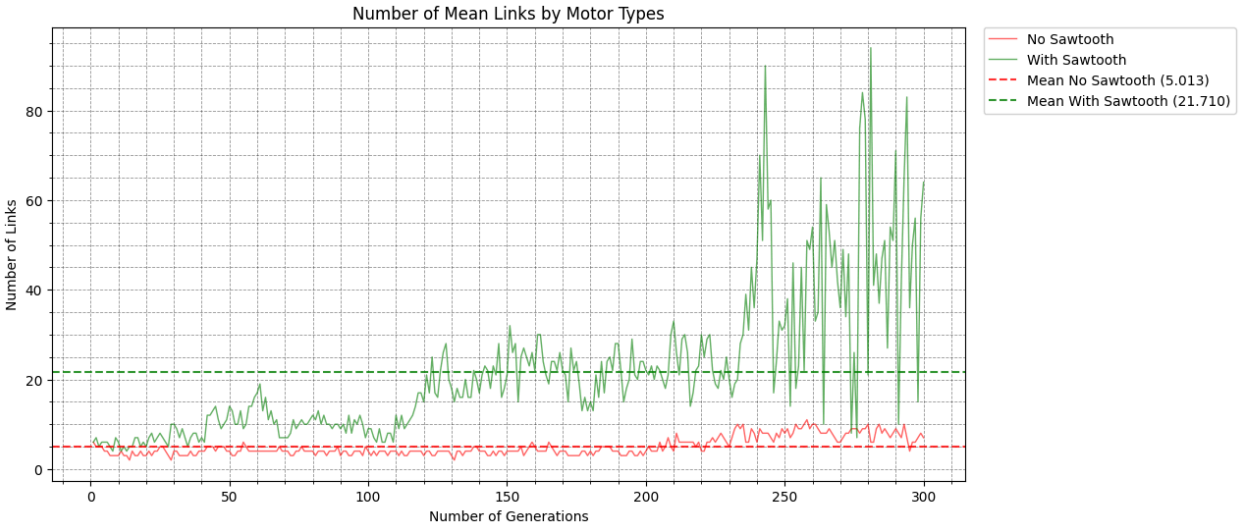
Population size	Best fitness	Mean fitness	Max links	Mean links
Standard	138.004	140.229	58	5.01
With Sawtooth	135.455	inf	1093	21.71

Immediately, we observe that the simulation with the sawtooth motor performs better across 300 generations. The best fitness value drops steadily, indicating that the creatures were able to consistently mutate and evolve to achieve best fitness.

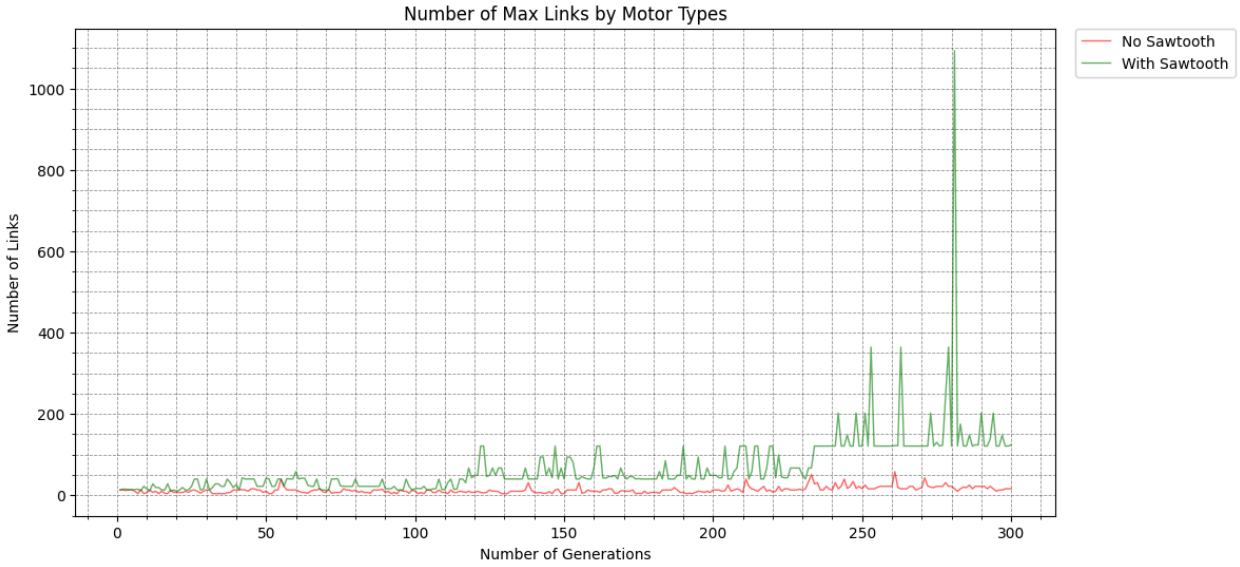
This observation tells us that adding a new motor type, specifically the sawtooth motor, is effective in achieving maximum fitness as it enables more effective movement controls.



Another observation is that the sawtooth motor introduces a high link count. In the graph, we can see that there is a greater increase in the number of mean links with the introduction of the sawtooth motor. After 230 generations, we see great fluctuations in the mean number of links.



At the same time, the maximum number of links sees a significant increase, up to 1093 links in generation 281. However, the increase in mean and maximum number of links does not correlate with significant improvements in best fitness.



However, the extreme complexity provides little additional benefit. After generation 230, despite the dramatic fluctuations and significant increase in the number of links, there is little improvement in the best fitness value. This means that the many of the added links may be non-functional, and is the reason why the mean fitness value returns ‘inf’ as it is computationally expensive to simulate.

Conclusion

Across all configurations, increasing the population size to 50 allows the simulation to achieve the best fitness at a value of 132~ after 300 generations, compared to the baseline performance value of 139.104.

We discover that increasing parameter values generally allow for better fitness values. However, increasing population size would significantly impact computational power and time needed to run the simulations. It is also found that the mutation parameters play complementary roles: grow mutation allows for exploration, while point and shrink mutations enables fine-tuning and reducing bloat. Introducing new motor types allows for better exploration, but needs limitations.

To balance creature performance and computational stability, it is recommended to introduce a hard constraint for the maximum number of links allowed for any given creature.

This project is limited by machine power. To achieve maximum fitness, the genetic algorithm would need to run for a greater number of generations, with greater population sizes. Genetic algorithms also rely on randomness to create creatures and generate movement. As such, it takes much longer to converge to the maximum fitness value of 0. These factors are ultimately why the creatures in this project were unable to achieve maximum fitness.